

HOSPITAL MANAGEMENT SYSTEM

Technical & Functional Documentation

Version 1.0 · March 2026

Document Type	Combined Technical & Functional Reference
Project	Hospital Management System (HMS)
Architecture	Microservices — Spring Boot 3.5 · Angular 21 · MySQL 8
Security	JWT Bearer Token · Spring Security · Role-Based Access Control
Purpose	Explain system design, capabilities and workflows to the review team
Author	Het Kachhiya
Date	March 2026

Table of Contents

1 Executive Summary

2 System Architecture

- 2.1 High-Level Overview
- 2.2 Microservice Breakdown
- 2.3 API Gateway & Routing
- 2.4 Service Discovery (Eureka)
- 2.5 Cross-Service Communication

3 Security Architecture

- 3.1 JWT Authentication Flow
- 3.2 Token Design & Claims
- 3.3 Multi-Layer Authorisation
- 3.4 Multi-Tenancy Model

4 Data Model

- 4.1 Database Overview
- 4.2 Entity Relationship Diagram
- 4.3 UML Class Diagrams
- 4.4 Full Table Reference

5 Functional Modules

- 5.1 User & Hospital Management
- 5.2 Patient Registration
- 5.3 Episode Management
- 5.4 Appointment Scheduling
- 5.5 Clinical Consultation
- 5.6 Laboratory Workflow
- 5.7 Dashboard & Analytics

6 User Roles & Access Control

7 API Reference

8 Frontend Architecture

9 Workflow Diagrams

10 Deployment & Configuration

11 Glossary

1. Executive Summary

The Hospital Management System (HMS) is a production-grade, cloud-ready web application that fully digitalises hospital operations. It replaces fragmented paper-based or siloed workflows with a single integrated platform spanning patient intake, clinical care delivery, laboratory diagnostics, and management analytics.

The system is built on a microservices architecture — four independently deployable Spring Boot services sit behind an API Gateway, all discovered by a Netflix Eureka registry. An Angular 21 single-page application (SPA) provides a role-driven user experience for eight distinct staff and patient personas.

1.1 What Problem Does HMS Solve?

Pain Point	How HMS Solves It
Manual paper registers for patients	Digital patient registration with Aadhaar linkage; searchable records
Phone-based appointment booking	Doctor slot calendar, real-time conflict checking, instant booking
Disconnected doctor notes	Structured consultation records linked to episodes and encounters
Handwritten prescriptions	Digital prescriptions attached to every consultation
Verbal lab order communication	Tracked lab orders from doctor request to final signed report
No visibility for management	Role-specific dashboards with live KPIs and Chart.js visualisations
Single hospital, single system	Multi-tenant: each hospital isolated by hospitalId in every query

1.2 Core Capabilities at a Glance

- ▶ **Multi-Hospital Tenancy:** Complete data isolation per hospital; super admin governs all hospitals
- ▶ **8 User Roles:** Superadmin, Hospital Admin, Doctor, Nurse, Front Desk, Lab Technician, Pharmacist, Patient
- ▶ **Clinical Episode Model:** Groups all visits for one health concern; full encounter and consultation history
- ▶ **Lab Workflow:** 5-stage tracked pipeline: ORDERED → BOOKED → SAMPLE_COLLECTED → IN_PROGRESS → COMPLETED
- ▶ **Stateless JWT Security:** No server sessions; all identity and role context travels inside signed tokens
- ▶ **Microservice Architecture:** Independent scaling, deployment, and ownership per domain
- ▶ **Real-time Analytics:** Role-personalised dashboards with trend charts, appointment funnels, staff distribution

1.3 Technology Summary

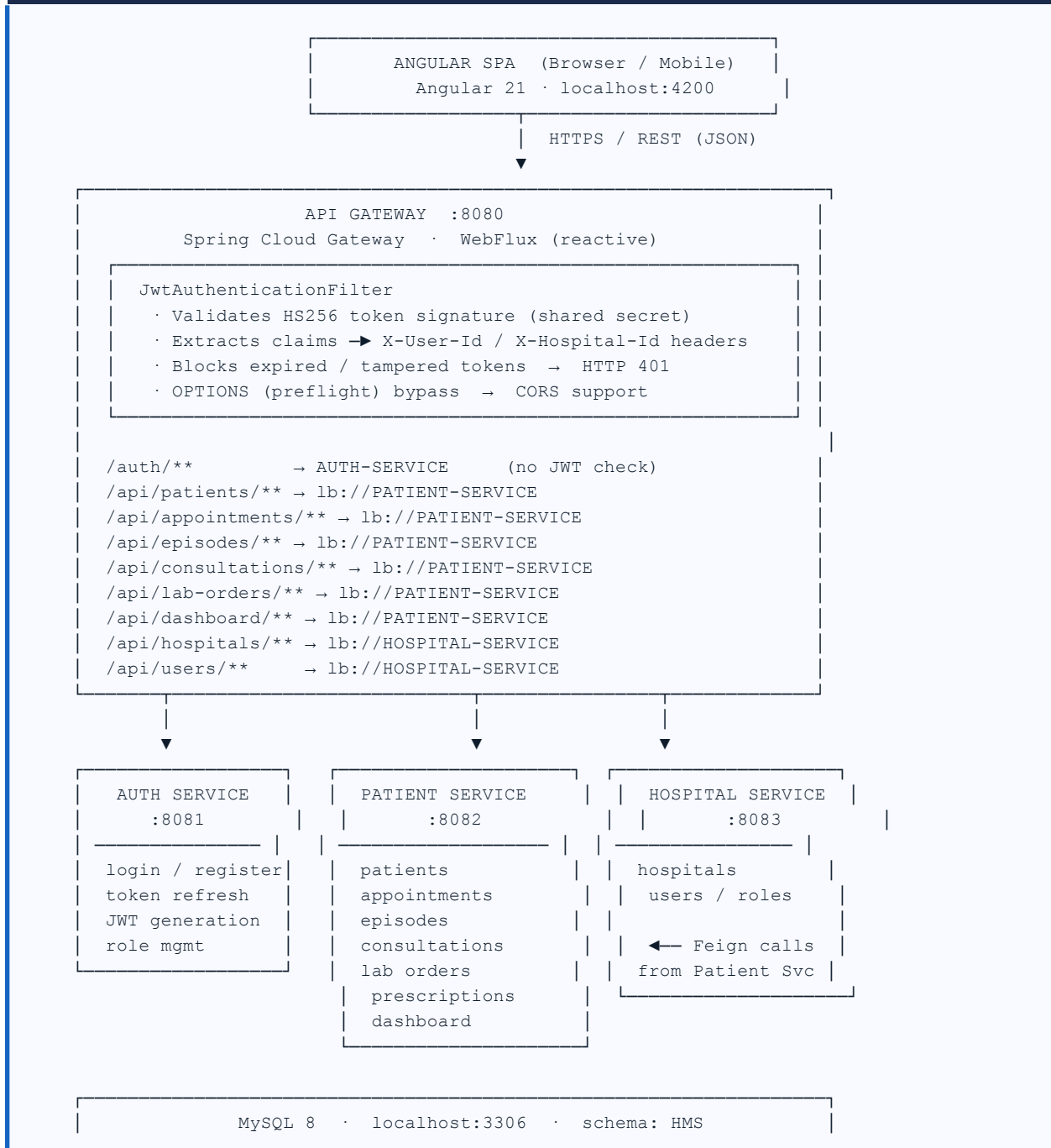
Layer	Technology	Version / Detail
Frontend	Angular + SCSS	Angular CLI 21.1.4 · Standalone components · Chart.js
API Gateway	Spring Cloud Gateway (WebFlux)	Reactive, non-blocking · JWT filter · CORS · lb:// routing
Auth Service	Spring Boot + Spring Security	JJWT 0.11.5 · BCrypt · Eureka client · MySQL
Patient Service	Spring Boot + JPA/Hibernate	Patients, appointments, episodes, labs, dashboard
Hospital Service	Spring Boot + JPA/Hibernate	Hospitals, users, roles · OpenFeign client
Service Registry	Netflix Eureka Server	Port 8761 · All services self-register
Database	MySQL 8	Single schema (HMS) · Hibernate DDL auto-update
Runtime	Java 21 / Node.js ≥ 18	Maven 3.x · npm · Angular CLI

2. System Architecture

2.1 High-Level Overview

HMS follows a microservices architecture pattern. Every HTTP request from the browser passes through the API Gateway — the sole public entry point. The gateway authenticates the request, enriches it with user-context headers, and routes it to the correct backend service, which is resolved by name through Eureka.

Figure 1 · High-Level System Architecture



```
users roles user_roles hospitals patients episodes
encounters appointments consultations prescriptions
lab_orders lab_order_tests lab_samples lab_reports
```

All JVM services ← EUREKA SERVER :8761 (Netflix OSS) → register & discover

2.2 Microservice Breakdown

Service	Port	Eureka Name	Responsibility
Eureka Server	8761	EUREKA-SERVER	Service registry — all services register here at startup
API Gateway	8080	API-GATEWAY	Single entry point, JWT validation, routing, CORS, load balancing
Auth Service	8081	AUTH-SERVICE	Identity management: login, register, token issuance & refresh
Patient Service	8082	PATIENT-SERVICE	Core clinical domain: patients, appointments, episodes, labs, analytics
Hospital Service	8083	HOSPITAL-SERVICE	Hospital & user management; exposes Feign endpoints for Patient Service
Angular Frontend	4200	N/A (browser)	SPA — role-driven UI, communicates exclusively with API Gateway

2.3 API Gateway Route Details

The API Gateway uses Spring Cloud Gateway route predicates to match URL patterns and apply the `JwtAuthenticationFilter` to all protected routes. The gateway performs load-balanced forwarding using `lb://` scheme resolved via Eureka.

URL Prefix	Target Service	JWT Required	Notes
/auth/login	AUTH-SERVICE	No	Open — no token needed
/auth/register	AUTH-SERVICE	No	Open — new user creation
/auth/refresh	AUTH-SERVICE	No	Open — token renewal
/api/patients/**	PATIENT-SERVICE	Yes	Multi-tenant by hospitalId
/api/appointments/**	PATIENT-SERVICE	Yes	Role-filtered results

URL Prefix	Target Service	JWT Required	Notes
/api/episodes/**	PATIENT-SERVICE	Yes	Clinical episode management
/api/consultations/**	PATIENT-SERVICE	Yes	Doctor consultation submission
/api/lab-orders/**	PATIENT-SERVICE	Yes	Full lab workflow pipeline
/api/dashboard/**	PATIENT-SERVICE	Yes	Role-specific analytics endpoints
/api/hospitals/**	HOSPITAL-SERVICE	Yes	Admin-only hospital management
/api/users/**	HOSPITAL-SERVICE	Yes	User & role management

2.4 Service Discovery — Eureka

Netflix Eureka Server runs on port 8761. Every Spring Boot microservice registers with Eureka on startup, sending its hostname, port, application name, and health status. The API Gateway uses the `lb://` prefix (Spring Cloud LoadBalancer) to resolve logical names like `PATIENT-SERVICE` to actual network addresses at request time. This means new service instances can be added without changing any gateway configuration.

How Eureka Routing Works

1. Patient Service starts → registers as `PATIENT-SERVICE` at `http://localhost:8082`
2. API Gateway receives request for `/api/patients/search`
3. Gateway resolves `lb://PATIENT-SERVICE` → looks up Eureka registry
4. Eureka returns `http://localhost:8082` (or multiple instances for load balancing)
5. Gateway forwards the request, stripping the `lb://` prefix

2.5 Cross-Service Communication

Patient Service calls Hospital Service using Spring Cloud OpenFeign — a declarative REST client that generates HTTP call code from interface annotations. This is used to enrich clinical data with user/doctor names without Patient Service owning the users table.

Figure 2 · Feign Client Cross-Service Call

Patient Service needs doctor name for Episode response

```
EpisodeServiceImpl.getEpisodeById(episodeId)
```

```
|  
├─ Load Episode from episodes table (local)
```

```
├─ Load Encounters from encounters table (local)
```

```
├─ Collect distinct doctorIds from encounters
```

```
└─ UserFeignClient.getUserByIds(doctorIds) → GET /api/users/by-ids?ids=3,7
```

```
|  
HOSPITAL-SERVICE responds with  
[{userId:3, firstName:'Priya',
```

```
lastName:'Sharma'},
```

```
{userId:7, firstName:'Rohan',
```

```
lastName:'Mehta'}]
```

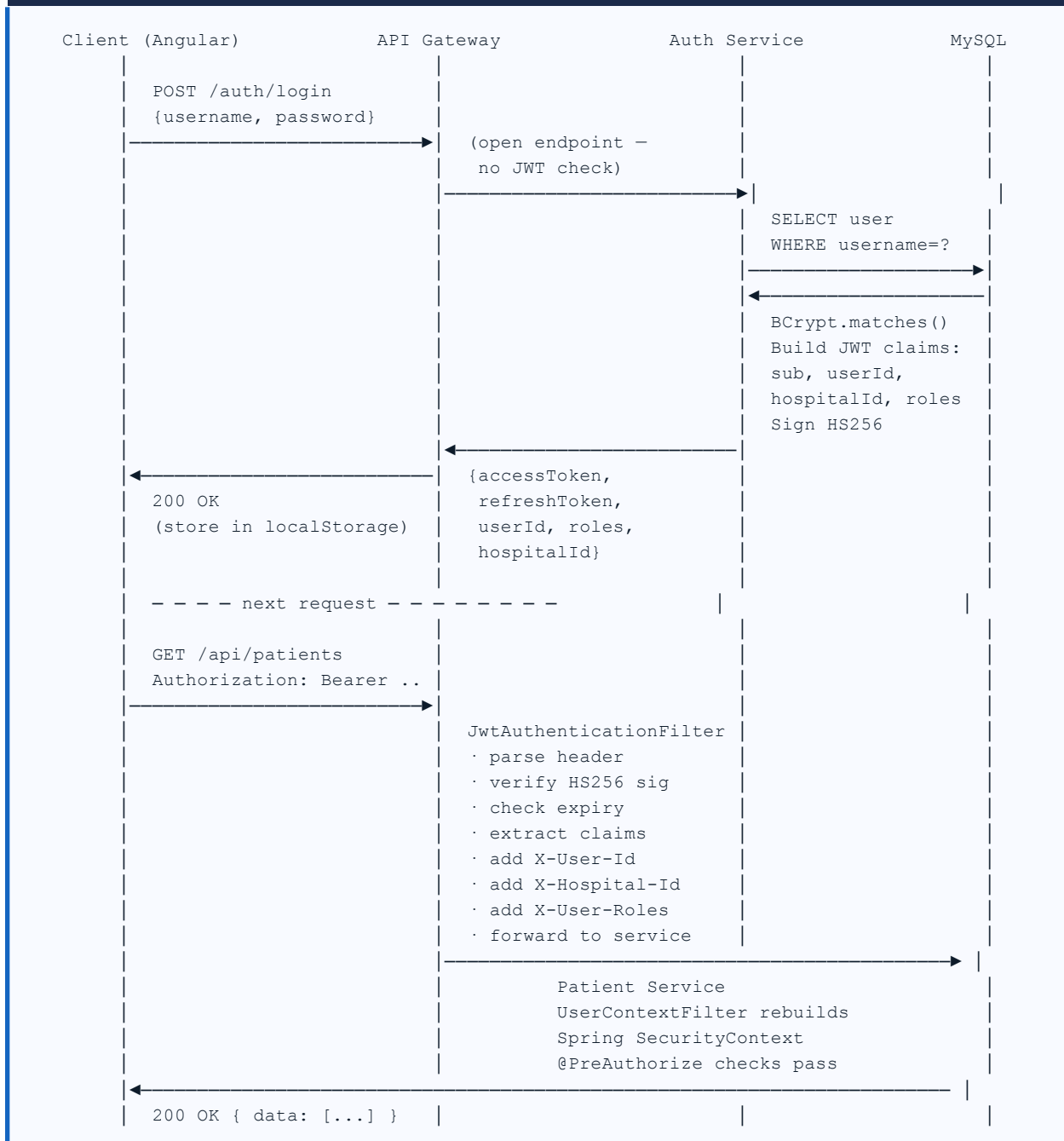
```
|  
Patient Service builds  
doctorId → name map  
and enriches EncounterDtos
```


3. Security Architecture

3.1 JWT Authentication Flow

HMS uses stateless JSON Web Token (JWT) authentication. No server-side sessions are stored. The Auth Service issues tokens; every subsequent request is verified by the API Gateway without touching a database, giving high throughput with no session overhead.

Figure 3 · JWT Authentication Sequence Diagram



3.2 Token Design & Claims

Tokens are signed with HMAC-SHA256 using a 256-bit secret shared between Auth Service and API Gateway. The secret is configured in `application.yaml` on both services. Both access and refresh tokens have a 24-hour lifetime in the current configuration.

JWT Claim	Type	Value Example	Purpose
sub	String	john.doe	Username — identity anchor for token refresh
userId	Integer	42	Forwarded as X-User-Id; used in audit fields & ownership checks
hospitalId	Integer	7	Forwarded as X-Hospital-Id; enforces multi-tenant isolation
roles	List<String>	[ROLE_DOCTOR]	Drives @PreAuthorize annotations and Angular navigation menus
email	String	john@hospital.com	Included in auth response for frontend display
userName	String	John	First name for personalised dashboard greeting message
iat	Epoch ms	1710000000000	Issued-at timestamp — tracks when token was created
exp	Epoch ms	1710086400000	Expiry timestamp — token rejected after this time

3.3 Multi-Layer Authorisation

Authorisation is enforced at four independent layers. An attacker who bypasses one layer still hits the next. This defence-in-depth approach means no single misconfiguration can expose data.

Layer	Where	Mechanism
Layer 1 — Transport	API Gateway	JwtAuthenticationFilter validates token signature and expiry on every non-open request. Returns HTTP 401 immediately if invalid.
Layer 2 — Context	Downstream Services	UserContextFilter (OncePerRequestFilter) reads X-User-Id, X-Hospital-Id, X-User-Roles headers and rebuilds Spring SecurityContext, giving each request a proper Authentication object.
Layer 3 — Method	Controllers	@PreAuthorize('hasRole(...)') annotations on each endpoint. A doctor can't call a Lab Technician-only endpoint even with a valid token.
Layer 4 — Frontend	Angular App	authGuard blocks navigation if localStorage has no token. authRole guard checks the role key before activating restricted routes.

3.4 Multi-Tenancy Model

Each hospital is a separate tenant. Data isolation is architectural — it is not achievable by misconfiguration — because hospitalId is extracted from the cryptographically signed JWT, not from a user-supplied parameter.

Figure 4 · Multi-Tenancy Enforcement

```

Doctor at Hospital A logs in
├─ JWT contains: { hospitalId: 1, userId: 42, roles: [ROLE_DOCTOR] }
├─ GET /api/patients
│   └─ PatientController reads X-Hospital-Id = 1 (from gateway header)
│       └─ Query: SELECT * FROM patients WHERE hospital_id = 1
│           └─ Hospital B patients (hospital_id = 2) are structurally unreachable
├─ GET /api/appointments
│   └─ AppointmentController reads hospitalId from header
│       └─ Query always filtered: WHERE hospital_id = {X-Hospital-Id}
└─ Cannot forge hospitalId because JWT is signed — any tampering breaks
    the HS256 signature and the gateway rejects the request with 401

```

4. Data Model

4.1 Database Overview

All five services share a single MySQL 8 schema named HMS. Inter-service foreign key relationships are maintained through application-level IDs (e.g. patientId stored as an Integer column in the episodes table) rather than database-level foreign key constraints. This is a deliberate microservices pattern: it allows each service to own and evolve its tables independently.

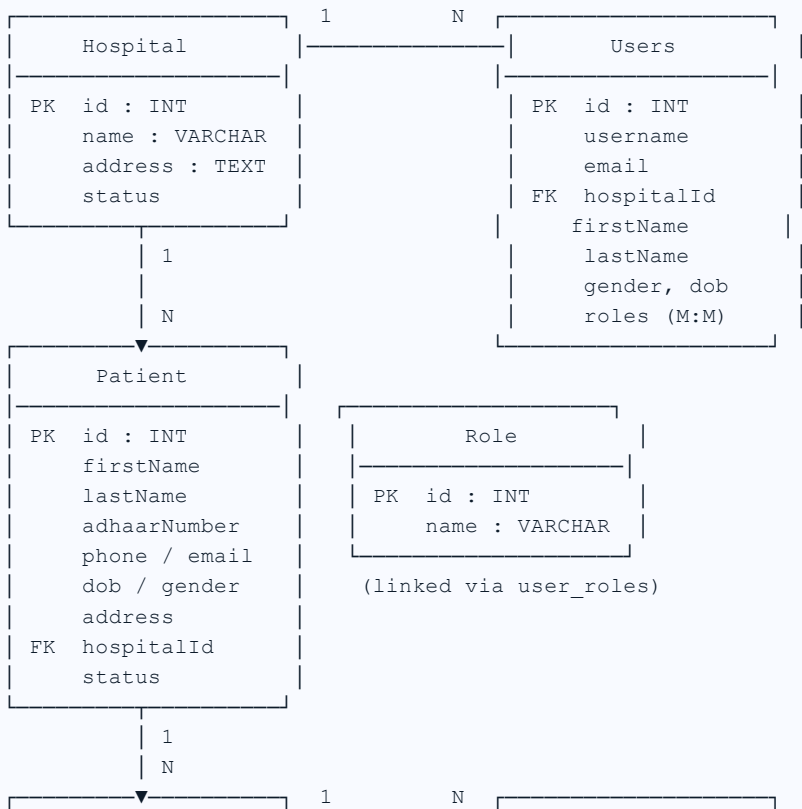


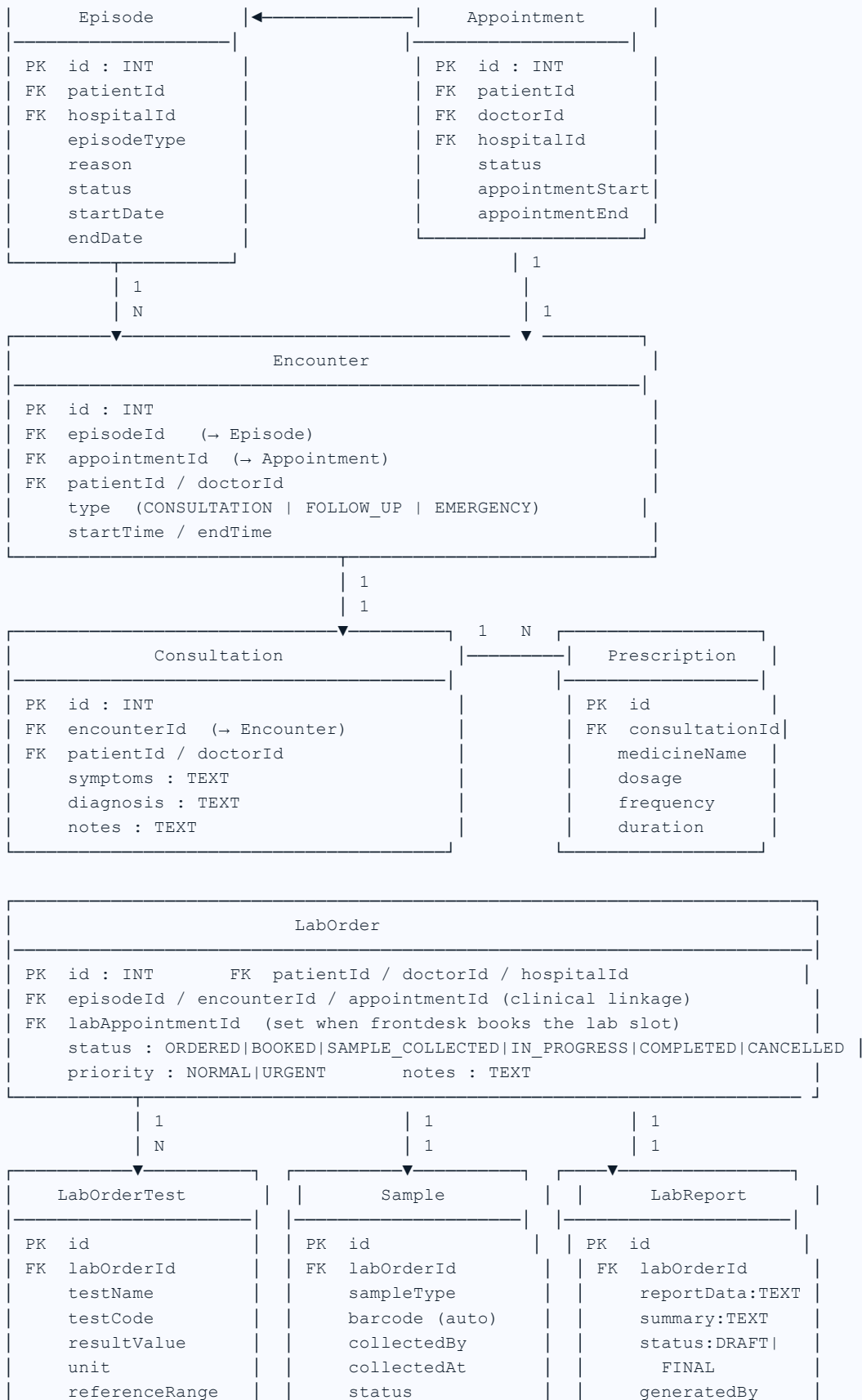
Important Design Note

There are no FOREIGN KEY constraints across service boundaries in the database.
Patient Service stores doctorId as a plain Integer — it never joins to the users table.
Name resolution is done at the application layer via Feign calls to Hospital Service.
This decoupling means services can be extracted to separate databases in the future without schema changes.

4.2 Entity Relationship Diagram

Figure 5 · Entity Relationship Diagram — HMS Database

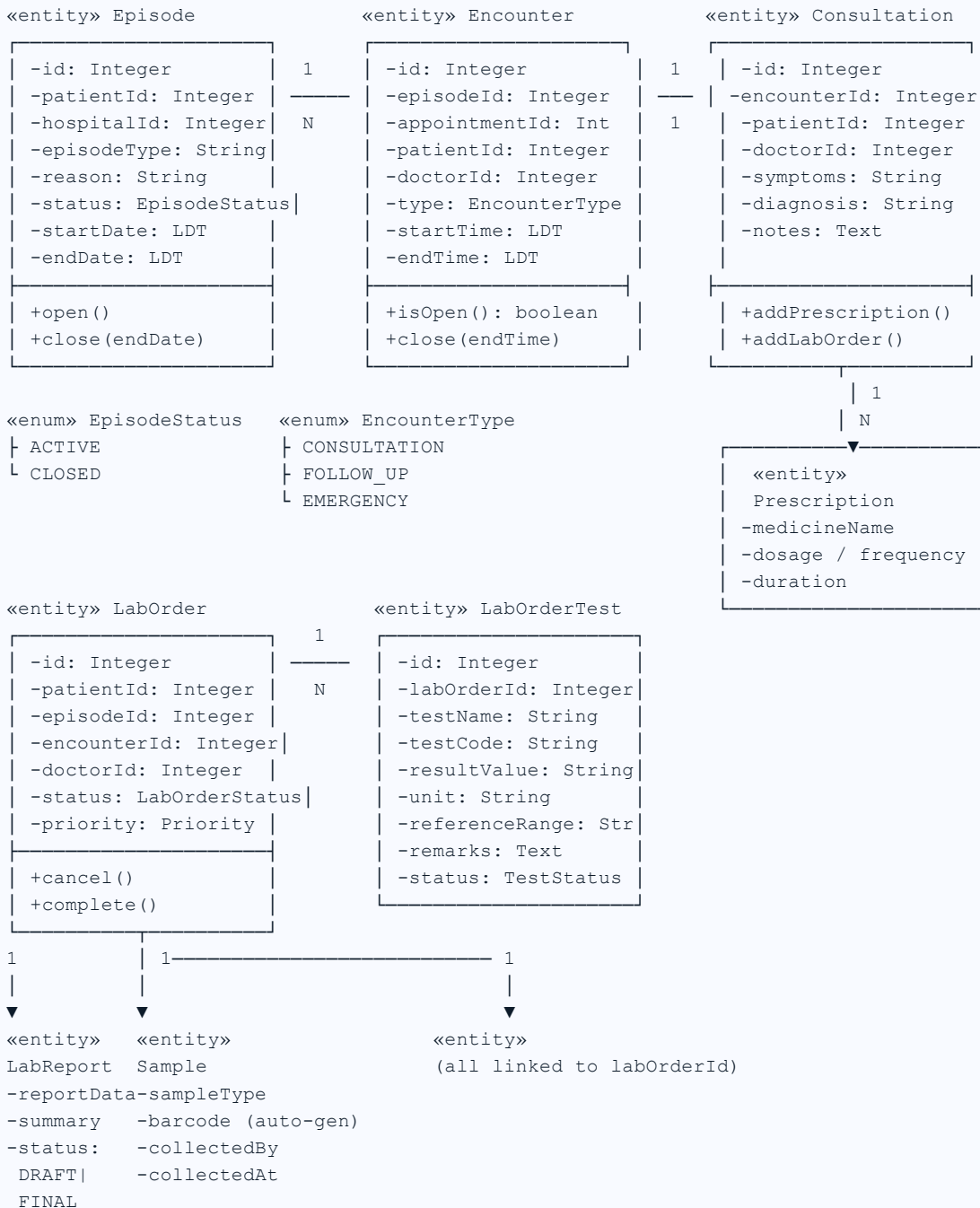




remarks		generatedAt
status:PENDING		
IN_PROGRESS		
COMPLETED		

4.3 UML Class Diagram — Clinical Domain

Figure 6 · UML Class Diagram — Clinical Domain



4.4 Full Table Reference

Table	Service Owner	Key Columns
users	Auth Service	id PK · username UNIQUE · email UNIQUE · hospitalId FK · password (BCrypt) · firstName · lastName · status · createdAt
roles	Auth Service	id PK · name (e.g. ROLE_DOCTOR, ROLE_NURSE)
user_roles	Auth Service	user_id FK · role_id FK (join table for ManyToMany)
hospitals	Hospital Service	id PK · name · address · status · createdAt
patients	Patient Service	id PK · firstName · lastName · adhaarNumber · phone · email · dob · gender · address · city · state · pincode · hospitalId · status
episodes	Patient Service	id PK · patientId FK · hospitalId · episodeType · reason · status (ACTIVE CLOSED) · startDate · endDate · createdBy · updatedBy
encounters	Patient Service	id PK · episodeId FK · appointmentId FK · patientId · doctorId · type (CONSULTATION FOLLOW_UP EMERGENCY) · startTime · endTime
appointments	Patient Service	id PK · patientId · doctorId · hospitalId · status (BOOKED CHECKED_IN IN_CONSULTATION COMPLETED CANCELLED) · appointmentStart · appointmentEnd · createdBy
consultations	Patient Service	id PK · encounterId FK (OneToOne) · patientId · doctorId · symptoms TEXT · diagnosis TEXT · notes TEXT
prescriptions	Patient Service	id PK · consultationId FK · medicineName · dosage · frequency · duration
lab_orders	Patient Service	id PK · patientId · episodeId · encounterId · appointmentId · labAppointmentId · doctorId · hospitalId · status · priority · notes · createdBy · updatedBy
lab_order_tests	Patient Service	id PK · labOrderId FK · testName · testCode · resultValue · unit · referenceRange · remarks · status (PENDING IN_PROGRESS COMPLETED)
lab_samples	Patient Service	id PK · labOrderId FK (OneToOne) · sampleType · sampleNotes · barcode · collectedBy · collectedAt · status
lab_reports	Patient Service	id PK · labOrderId FK (OneToOne) · reportData TEXT · reportUrl · summary TEXT · status (DRAFT FINAL) · generatedBy · generatedAt · finalizedAt

5. Functional Modules

This section describes each functional module from the perspective of what a user can do, which screens are involved, and how the system handles the request behind the scenes.

5.1 Module: User & Hospital Management



What This Module Does

Allows Super Admin to onboard hospitals and create staff accounts with specific roles.

Hospital Admin can add doctors, nurses, front desk, and lab technicians to their hospital.

All user accounts carry a hospitalId which determines which data they can see and modify.

Functional Flows

Creating a Hospital (Super Admin)

1. Super Admin navigates to /hospitals and clicks 'Add Hospital'
2. Fills hospital name, address, and saves — POST /api/hospitals
3. Hospital appears in the list with an auto-assigned ID
4. Super Admin then creates users for that hospital, assigning it by hospitalId

Creating a User / Staff Account

5. Admin navigates to /users and opens the User Form modal
6. Fills firstName, lastName, email, username, phone, DOB, gender
7. Selects the hospital (from dropdown — Super Admin sees all, Hospital Admin sees their own)
8. Checks one or more role checkboxes (Doctor, Nurse, Front Desk, etc.)
9. System POSTs to /api/users — password auto-generated or set at registration
10. New user can log in with the assigned username and appears in the hospital's user list

Key Business Rules

- A user can hold multiple roles simultaneously (ManyToMany relationship)
- hospitalId on a user determines their data scope — cannot be changed after creation without admin action
- Super Admin can create users across any hospital; Hospital Admin can only create within their hospital

5.2 Module: Patient Registration



What This Module Does

Records a new patient's demographic and contact information into the system.

Every patient is tied to a hospital via hospitalId — they cannot cross-hospital access.
Aadhaar number serves as the unique national ID for searching and linking patients to episodes.

Patient Data Captured

Field Group	Fields
Identity	firstName, lastName, dateOfBirth, gender
Contact	phone, email
Address	address (line), city, state, pincode
Government ID	adhaarNumber (12-digit, validated)
System	hospitalId (auto from logged-in user's JWT), status (ACTIVE/INACTIVE), createdBy

Functional Flow

11. Front Desk opens /patients and clicks 'Register Patient'
12. Patient Form modal opens — fills all required fields including Aadhaar
13. System validates Aadhaar is exactly 12 digits
14. POST /api/patients — hospitalId injected from X-Hospital-Id header (not user-supplied)
15. Patient record saved; returned to list with new patient ID
16. Patient can now be searched by name or Aadhaar for episode and appointment creation

Search & Lookup

- Search by partial name (firstName + lastName contains) — paginated results
- Autocomplete search in appointment and episode forms — type-ahead by name or Aadhaar
- Count endpoint used by dashboards: /api/patients/count?hospitalId=X

5.3 Module: Episode Management



What This Module Does

An Episode is the top-level clinical grouping — it represents one 'case' or 'visit reason'.
All appointments, encounters, consultations, and lab orders for a single health concern are grouped under one episode.
This gives doctors a complete clinical timeline for every visit related to the same condition.

Episode Hierarchy Example

Figure 7 · Episode Hierarchy — Clinical Timeline Example

```

Episode #47 - 'Persistent Fever' - Status: ACTIVE
├── Encounter #1 — Appointment #88 (COMPLETED, 10 Jan 2026)
│   ├── Consultation #12
│   │   ├── Symptoms: 'High fever 103°F, chills, body ache'
│   │   ├── Diagnosis: 'Suspected viral fever - investigate'
│   │   ├── Prescriptions: [Paracetamol 500mg, 3x/day, 5 days]
│   │   └── LabOrder #5 - [CBC, LFT, CRP] — Status: COMPLETED
│   │       ├── Sample: Blood (Venous) · barcode LAB-5-17101234
│   │       └── LabReport: FINAL (CBC normal, LFT mildly elevated)
└── Encounter #2 — Appointment #102 (COMPLETED, 20 Jan 2026)
    ├── Consultation #19
    │   ├── Symptoms: 'Fever resolved, mild fatigue'
    │   ├── Diagnosis: 'Recovery - Viral fever'
    │   ├── Prescriptions: [Multivitamin, 1x/day, 30 days]
    │   └── Episode closed by doctor on completion

```

Functional Flow

17. Front Desk searches for patient by name or Aadhaar in the Episode form
18. Selects patient → fills episodeType and reason → POST /api/episodes
19. Episode created with status ACTIVE; appears in episodes list
20. When booking an appointment, Front Desk selects this episode — link is made
21. Doctor may close the episode during consultation by ticking 'Close Episode'
22. Episode Detail page (/episode/details/:id) shows full encounter and consultation history

Business Rules

- One patient can have multiple open episodes simultaneously (e.g., one for fever, one for fracture)
- Episode search uses Aadhaar number as the lookup key — type-ahead from patient data
- Closing an episode sets endDate = now and status = CLOSED
- Episode Detail view fetches encounters and enriches each with doctor name via Feign call

5.4 Module: Appointment Scheduling



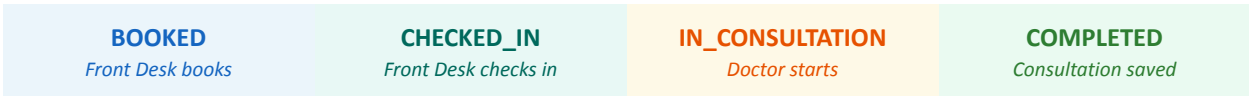
What This Module Does

Allows Front Desk to book appointments for patients with specific doctors.

Displays a visual calendar of the doctor's existing appointments to prevent time conflicts.

Booking an appointment with an episode automatically creates an Encounter — the clinical context for that visit.

Appointment Status Lifecycle



Any appointment may be cancelled (status = CANCELLED) at any point before COMPLETED by authorised roles.

Functional Flow

- 23. Front Desk opens /appointments and clicks 'New Appointment'
- 24. Searches for patient by name → selects from dropdown
- 25. Searches for doctor by name → selects from dropdown
- 26. Selects episode (optional — patient's open episodes appear in dropdown by Aadhaar)
- 27. Picks appointment date — system loads doctor's existing appointments for that day on a visual calendar
- 28. Selects available time slot → sets start and end time → selects encounter type
- 29. POST /api/appointments — if episodeId is provided, an Encounter is also created transactionally
- 30. Doctor sees appointment in their queue; Front Desk checks patient in when they arrive

Doctor Slot Availability

- Appointment form loads the doctor's existing appointments for the selected date
- Occupied slots are shown on the time picker — prevents double booking
- Count endpoint gives Today's breakdown: total / booked / checked-in / completed / cancelled

5.5 Module: Clinical Consultation & Prescription



What This Module Does

Allows doctors to record the clinical findings from a patient encounter.
A consultation captures symptoms, diagnosis, clinical notes, and issues prescriptions.
Submitting a consultation triggers a cascade: encounter closes, appointment completes, episode optionally closes.

Consultation Submission — Transactional Cascade

Figure 8 · Consultation Submission — Transactional Cascade

```
Doctor submits Consultation form
|
├─ Step 1: Validate Encounter exists and is open
|
```

```

└─ Step 2: Save Consultation record
    { encounterId, patientId, doctorId, symptoms, diagnosis, notes }

└─ Step 3: If closeEpisode = true
    └─ Episode.status = CLOSED, Episode.endDate = now()

└─ Step 4: Encounter.endTime = now()    (encounter is closed)

└─ Step 5: Appointment.status = COMPLETED

└─ Step 6: Save Prescriptions[]
    Each: { consultationId, medicineName, dosage, frequency, duration }

└─ Step 7: Save LabOrders[]    (if doctor ordered any tests)
    Each: { patientId, episodeId, encounterId, doctorId, tests[], priority }
    Initial LabOrder.status = ORDERED

```

The entire flow is wrapped in @Transactional(rollbackFor = Exception.class)
 If any step fails, ALL changes are rolled back atomically.

What the Doctor Fills In

Field	Type	Description
Encounter	Dropdown	Select the encounter (linked to episode + appointment) for this consultation
Symptoms	Free text	Patient's reported symptoms — 'High fever 103°F, chills since 3 days'
Diagnosis	Free text	Doctor's clinical finding — 'Suspected viral fever, rule out typhoid'
Notes	Rich text	Additional clinical notes, examination findings, follow-up instructions
Prescriptions	Repeating	One or more: medicineName, dosage, frequency (1x/2x/3x), duration
Lab Orders	Repeating	One or more tests: testName, testCode, priority (NORMAL/URGENT)
Close Episode	Checkbox	If checked, marks the episode as resolved/closed on submission

5.6 Module: Laboratory Workflow



What This Module Does

Manages the full lifecycle of a diagnostic lab test from doctor order to patient report.

Five sequential stages enforced by the system — each stage can only be triggered in order.
 Produces a FINAL lab report containing all test results, reference ranges, and technician summary.

Lab Order Status Flow



Any order can be CANCELLED by Doctor, Front Desk, or Hospital Admin before it reaches COMPLETED.

Step-by-Step Lab Workflow

Figure 9 · Laboratory Workflow — Stage-by-Stage

```

STAGE 1 — Doctor Orders Tests (status: ORDERED)
├─ Doctor submits consultation and includes lab tests
├─ POST /api/lab-orders
├─ LabOrder created with status = ORDERED
└─ LabOrderTest rows created (one per test, status = PENDING)

STAGE 2 — Front Desk Books Lab Appointment (status: BOOKED)
├─ Front Desk opens /lab-orders, sees orders with status = ORDERED
├─ Clicks 'Book' → selects date and time slot
├─ POST /api/lab-orders/{id}/book
├─ Lab Appointment created (type = LAB), linked via labAppointmentId
└─ LabOrder.status = BOOKED

STAGE 3 — Lab Technician Collects Sample (status: SAMPLE_COLLECTED)
├─ Lab Tech opens /lab-orders/{id}/collect-sample
├─ Selects sample type (Blood Venous / Urine / Stool / Swab / etc.)
├─ Adds collection notes (e.g. 'Fasting sample')
├─ POST /api/lab-orders/{id}/collect-sample
├─ Sample record created with auto-generated barcode: LAB-{id}-{timestamp}
└─ LabOrder.status = SAMPLE_COLLECTED

STAGE 4 — Lab Tech Starts Processing (status: IN_PROGRESS)
├─ POST /api/lab-orders/{id}/start-processing
├─ All LabOrderTests.status = IN_PROGRESS
└─ LabOrder.status = IN_PROGRESS

STAGE 4b — Lab Tech Enters Results
├─ POST /api/lab-orders/{id}/results
├─ Payload: [{ testId, resultValue, unit, referenceRange, remarks }]
└─ Each LabOrderTest updated with results, status = COMPLETED

STAGE 5 — Lab Tech Generates Report (status: COMPLETED)
├─ Precondition: ALL LabOrderTests must be COMPLETED (validated server-side)
├─ POST /api/lab-orders/{id}/generate-report
├─ Body: { summary: 'All markers within range except LFT...' }
└─ LabReport created: status = FINAL, reportData = JSON of all results
  
```

```
└─ Lab Appointment.status = COMPLETED
└─ LabOrder.status = COMPLETED
```

Sample Type Options

The system supports the following sample types selectable during Stage 3:

Blood (Venous) · Blood (Capillary) · Urine (Spot) · Urine (24hr) · Stool · Sputum · Swab (Throat) · Swab (Nasal) · Swab (Wound) · CSF · Pleural Fluid · Other

5.7 Module: Dashboard & Analytics



What This Module Does

Each role receives a personalised dashboard on login with KPI stat cards and Chart.js charts. Data is served by dedicated role endpoints in the DashboardController — never shared across roles. Charts include: weekly appointment trend, monthly trend, hourly distribution, top diagnoses, staff distribution, appointment funnel.

Role	KPI Cards	Charts & Visualisations
Super Admin	Total hospitals, active hospitals, total users, total doctors, total patients, appointments today	System-wide user distribution pie
Hospital Admin	Total doctors, nurses, all staff, patients, appointments today, checked-in, consultations, open episodes	Staff distribution pie chart + appointment status bar chart
Doctor	Appointments today, checked-in, completed, consultations today, my patients, upcoming appointments	Weekly trend line, monthly trend bar, hourly distribution bar, top diagnoses horizontal bar
Front Desk	Total patients, appointments today, cancelled, completed today	Appointment status doughnut chart + status bar chart
Nurse	Assigned patients, ward summary	Placeholder — future ward analytics
Lab Technician	Orders by status: ordered, booked, in-progress, completed today	Lab pipeline summary
Pharmacist	Pending prescriptions, dispensed today	Placeholder — future pharmacy charts
Patient	Upcoming appointments, available lab reports	Personal health summary

6. User Roles & Access Control

6.1 Role Definitions

Role (Display)	Spring Role	Angular Key	Primary Responsibilities
Super Admin	ROLE_SUPER_ADMIN	superadmin	Global governance — hospitals, all users, system-wide reports, audit logs
Hospital Admin	ROLE_HOSPITAL_ADMIN	hospitaladmin	Manage own hospital's staff, patients, appointments, billing, inventory
Doctor	ROLE_DOCTOR	doctor	View appointments, record consultations, issue prescriptions, order labs
Nurse	ROLE_NURSE	nurse	Assigned patients, vitals entry, medication tracking, ward management
Front Desk	ROLE_FRONTDESK	frontdesk	Patient check-in, episode creation, appointment booking, lab scheduling
Lab Technician	ROLE_LAB_TECHNICIAN	labtechnician	Sample collection, test processing, result entry, report generation
Pharmacist	ROLE_PHARMACIST	pharmacist	Prescription fulfilment, inventory management, dispensing
Patient	ROLE_PATIENT	patient	Own health portal — appointments, lab reports, profile

6.2 Feature Access Matrix

✓ = Full access · R = Read-only · (blank) = No access

Feature	S.Admin	H.Admin	Doctor	Nurse	F.Desk	Lab Tech	Pharm.	Patient
Manage Hospitals	✓							
Manage Users	✓	✓						

Feature	S.Admin	H.Admin	Doctor	Nurse	F.Desk	Lab Tech	Pharm.	Patient
View All Patients	✓	✓	✓	✓	✓			
Register Patient	✓	✓			✓			
Create Episode	✓	✓	✓		✓			
Book Appointment	✓	✓			✓			
Check-in Patient	✓	✓			✓			
View Appointments	✓	✓	✓		✓			✓
Record Consultation			✓					
Issue Prescription			✓					
Order Lab Tests			✓					
Book Lab Appointment	✓	✓			✓			
Collect Sample						✓		
Enter Test Results						✓		
Generate Lab Report						✓		
View Lab Reports	✓	✓	✓		✓	✓		✓
Vitals / Ward Mgmt				✓				
Dispense Medication							✓	
Analytics Dashboard	✓	✓	✓	✓	✓	✓	✓	✓
Audit Logs	✓							

7. API Reference

All API calls pass through the API Gateway on port 8080. Protected endpoints require Authorization: Bearer <token> header. The gateway strips this header and forwards X-User-Id, X-Hospital-Id, X-Username, X-User-Roles to the downstream service.

7.1 Auth Service (:8081)

Method	Endpoint	Auth	Request Body / Params
POST	/auth/login	None	{ username, password } → { accessToken, refreshToken, userId, hospitalId, roles }
POST	/auth/register	None	{ username, email, password, firstName, lastName, hospitalId, roles[] }
POST	/auth/refresh	None	{ refreshToken } → new { accessToken }
GET	/auth/validate	None	?token=... → { valid: true/false }

7.2 Patient Service — Patients (:8082)

Method	Endpoint	Role	Description
GET	/api/patients	All clinical	List patients for hospital, paginated, filterable by name
GET	/api/patients/search	All clinical	Search by name or Aadhaar — used by appointment/episode forms
POST	/api/patients	FrontDesk, Admin	Register new patient
PUT	/api/patients/{id}	FrontDesk, Admin	Update patient details
GET	/api/patients/count	Admin	Count for dashboard KPI

7.3 Patient Service — Appointments

Method	Endpoint	Role	Description
GET	/api/appointments	Doctor, FrontDesk	List appointments filtered by role and hospital

Method	Endpoint	Role	Description
POST	/api/appointments	FrontDesk	Create appointment (+ Encounter if episodeId provided)
PUT	/api/appointments/{id}	FrontDesk, Doctor	Update appointment details
PATCH	/api/appointments/{id}/status	FrontDesk, Doctor	Change status (CHECKED_IN / COMPLETED / CANCELLED)
GET	/api/appointments/doctor/{id}	FrontDesk	Doctor's appointments for a date range — for slot picker
GET	/api/appointments/count	Doctor, FrontDesk	Count by status for dashboard metrics

7.4 Patient Service — Episodes & Consultations

Method	Endpoint	Role	Description
GET	/api/episodes	Doctor, FrontDesk	List episodes (paginated) for hospital
POST	/api/episodes	FrontDesk	Create episode for patient
GET	/api/episodes/{id}	All clinical	Get episode + encounters + doctor names (Feign enriched)
GET	/api/episodes/search	Doctor, FrontDesk	Search episodes by Aadhaar number
POST	/api/consultations	Doctor	Submit consultation + prescriptions + lab orders (transactional)
GET	/api/consultations/{id}	Doctor, FrontDesk	Get consultation with prescriptions

7.5 Patient Service — Lab Orders

Method	Endpoint	Role	Description
POST	/api/lab-orders	Doctor	Create lab order with test list
GET	/api/lab-orders	FrontDesk, Admin	List all orders (filterable by status)

Method	Endpoint	Role	Description
GET	/api/lab-orders/{id}	All lab roles	Get single order with tests, sample, report
POST	/api/lab-orders/{id}/book	FrontDesk, Admin	Book lab appointment → BOOKED
DELETE	/api/lab-orders/{id}	Doctor, FrontDesk	Cancel lab order
POST	/api/lab-orders/{id}/collect-sample	Lab Technician	Collect sample + barcode → SAMPLE_COLLECTED
POST	/api/lab-orders/{id}/start-processing	Lab Technician	Mark tests IN_PROGRESS
POST	/api/lab-orders/{id}/results	Lab Technician	Enter per-test results
POST	/api/lab-orders/{id}/generate-report	Lab Technician	Finalise report → COMPLETED

7.6 Hospital Service (:8083)

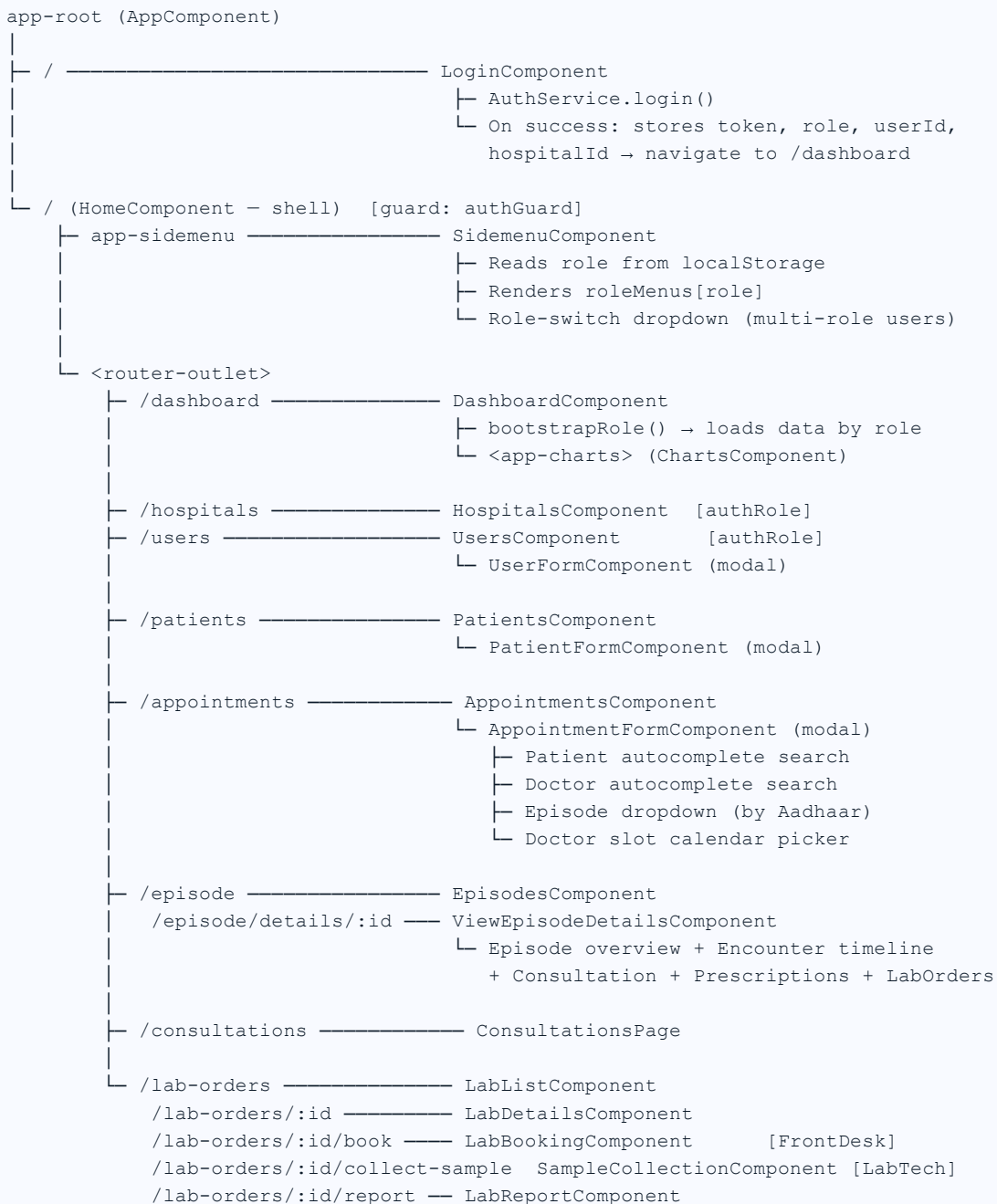
Method	Endpoint	Role	Description
GET	/api/hospitals	Super Admin	List all hospitals
POST	/api/hospitals	Super Admin	Create new hospital
GET	/api/hospitals/names	All admin	Get name/id list for dropdowns
GET	/api/users	Admin	List users, filterable by hospital
POST	/api/users	Admin	Create user and assign roles
GET	/api/users/count	Admin	Staff count (total, doctors, nurses) for dashboard
GET	/api/users/by-ids	Internal (Feign)	Batch user lookup for name enrichment
GET	/api/users/doctors	FrontDesk	Doctor list for appointment booking dropdown
GET	/api/roles	Admin	List all available system roles

8. Frontend Architecture

8.1 Angular Application Structure

The Angular 21 application uses standalone components exclusively — there are no NgModules. Each page and component imports only what it needs. The application uses lazy-loaded routes where possible and a central HomeComponent shell that hosts the sidebar navigation and router-outlet.

Figure 10 · Angular Component Tree & Route Map



8.2 Authentication State Management

There is no NgRx or state management library. Authentication state lives in `localStorage` and is accessed via `AuthService`. A `BehaviorSubject (user$)` broadcasts user changes to subscribed components.

localStorage Key	Value
jwt_token	Access token (Bearer) — sent as Authorization header on every API call
refresh_token	Refresh token — used to obtain new access token when expired
user_id	Numeric <code>userId</code> extracted from login response
hospital_id	Numeric <code>hospitalId</code> — used by components to filter data without extra API calls
user_role	Internal role key (e.g. 'doctor') — drives menu rendering and dashboard variant selection
user	JSON: { <code>userId</code> , <code>hospitalId</code> , <code>roles[]</code> , <code>username</code> } — used by sidemenu and header components

8.3 HTTP Interceptor Pattern

Angular's `HttpClient` is used directly in each service. The JWT token is read from `localStorage` and added to the Authorization header by each service's HTTP call. The API gateway then validates and strips this before forwarding downstream.



Future Improvement Note

The current implementation reads the JWT from `localStorage` inside each service call.

A production upgrade would introduce an `HttpInterceptor` that automatically attaches the Bearer header to every outgoing request, and a second interceptor that catches 401 responses and attempts token refresh.

8.4 Angular Services Reference

Service	Injected In	Responsibility
AuthService	All pages	<code>login()</code> , <code>logout()</code> , <code>getRole()</code> , <code>getToken()</code> , <code>mapRoleToInternal()</code> . Manages all <code>localStorage</code> auth state and emits <code>user\$</code> observable.
PatientService	Patients, Appts, Eps	CRUD, search by name/Aadhaar, count. Uses <code>hospitalId</code> from <code>localStorage</code> in queries.

Service	Injected In	Responsibility
AppointmentService	Appts, Dashboard	Create, update status, getDoctorAppointments() for slot calendar, count() for dashboard KPIs.
EpisodeService	Episodes, Appts	CRUD, getAll() paginated, search() by Aadhaar, getById() for detail view.
ConsultationService	Consultations	submit() — sends full consultation payload including prescriptions and lab orders.
LabService	Lab pages	All 8 lab endpoints: create, book, collectSample, startProcessing, enterResults, generateReport, getById, list.
DashboardService	Dashboard	getDoctorAnalytics(), getFrontdeskAnalytics(), getHospitalAdminAnalytics(), getSuperAdminAnalytics().
UserService	Users, Dashboard	CRUD, count() by hospital, getDoctors() for appointment dropdown.
HospitalService	Hospitals, UserForm	getAll(), getHospitalNames(), getHospitalNamesExceptSuperAdmin().
ToastService	All pages	Global notification system: success(), error(), info() — renders snackbar overlays.

9. Workflow Diagrams

9.1 Complete Patient Journey — All Actors

Figure 11 · Complete Patient Journey — All Actors & System Effects

ACTOR	ACTION	SYSTEM EFFECT
Front Desk	→ Register patient (name, Aadhaar, phone)	→ patients table
Front Desk	→ Create Episode (reason, type)	→ episodes table (ACTIVE)
Front Desk	→ Book Appointment (doctor, date, slot)	→ appointments (BOOKED) encounters table
Front Desk	→ Check-in patient on arrival	→ appointment → CHECKED_IN
Doctor	→ View queue, open consultation form	→ fetch encounter data
Doctor	→ Record symptoms, diagnosis, notes	→
Doctor	→ Add prescriptions (medicine, dosage)	→ (part of submission)
Doctor	→ Add lab tests (CBC, LFT...)	→ (part of submission)
Doctor	→ Click 'Submit Consultation' (optionally tick Close Episode)	→ consultations table prescriptions table lab_orders table (ORDERED) appointment → COMPLETED encounter closed episode → CLOSED (if ticked)
Front Desk	→ Open lab orders list, click Book select time slot	→ lab appointment created lab_orders → BOOKED
Lab Tech	→ Open order, click Collect Sample select sample type (Blood/Urine/...)	→ lab_samples table barcode: LAB-{id}-{ts} lab_orders →
SAMPLE_COLLECTED		
Lab Tech	→ Click Start Processing	→ lab_order_tests →
IN_PROGRESS		lab_orders → IN_PROGRESS
Lab Tech	→ Enter results for each test (resultValue, unit, reference range)	→ lab_order_tests with resultValue, unit, test status → COMPLETED
refRange		
Lab Tech	→ Write summary, Generate Report (all tests must be COMPLETED first)	→ lab_reports (FINAL) lab_orders → COMPLETED lab appointment →
COMPLETED		
Doctor	→ View episode details	→ episode detail page shows full timeline + report
Patient	→ View lab reports	

9.2 Appointment Booking with Encounter Creation

Figure 12 · Appointment + Encounter Creation Flow

```
POST /api/appointments
Body: { patientId, doctorId, hospitalId, appointmentStart, appointmentEnd,
       status:'BOOKED', episodeId (optional), type:'CONSULTATION' }

└─ AppointmentServiceImpl.createAppointment()
    └─ validateAppointment(): check no overlapping doctor slots
    └─ Save Appointment record → appointments table
        { patientId, doctorId, hospitalId, status:BOOKED,
          appointmentStart, appointmentEnd, createdBy }
    └─ if episodeId present:
        └─ Load Episode (validate it exists)
            └─ Save Encounter → encounters table
                { episodeId, appointmentId (saved.id), patientId,
                  doctorId, type:CONSULTATION, startTime:now() }

Result: one Appointment + one Encounter created atomically
The Encounter links Episode ↔ Appointment for the full clinical trail
```

9.3 API Gateway Request Processing

Figure 13 · API Gateway Request Processing Pipeline

```

Incoming request: GET /api/patients + Authorization: Bearer eyJ...
├─ JwtAuthenticationFilter.apply()
│   └─ 1. Check if path starts with /auth/* → if yes, skip filter
│       └─ 2. Check if method is OPTIONS → if yes, skip filter (CORS preflight)
│       └─ 3. Extract Authorization header
│           └─ If missing or not starting with 'Bearer ' → return 401
│       └─ 4. Extract token string (after 'Bearer ')
│       └─ 5. jwtUtil.validateToken(token)
│           └─ Parse with JJWT parserBuilder + signing key
│           └─ Verify HS256 signature
│           └─ Verify expiry (exp claim)
│           └─ If invalid → return 401 'Invalid or expired JWT token'
│       └─ 6. jwtUtil.extractAllClaims(token)
│           └─ Extract: userId, hospitalId, username, roles
│       └─ 7. Mutate request: add headers
│           └─ X-User-Id = claims.userId
│           └─ X-Hospital-Id = claims.hospitalId
│           └─ X-Username = claims.sub
│           └─ X-User-Roles = comma-separated roles
│       └─ 8. chain.filter(exchange) → forward to PATIENT-SERVICE
│           └─ UserContextFilter reads X-* headers
│               └─ Sets SecurityContextHolder with authorities
│                   └─ @PreAuthorize check passes
│                       └─ Controller method executes

```

10. Deployment & Configuration

10.1 Prerequisites

Requirement	Version	Notes
Java JDK	21 (Temurin)	Configured as 'temurin-24' in IntelliJ project settings
Maven	3.x	Build and run backend services: mvn spring-boot:run
Node.js	≥ 18	Required for Angular CLI and npm package management
Angular CLI	21.1.4	npm install -g @angular/cli@21.1.4 then ng serve
MySQL Server	8.0+	Running on localhost:3306. Create schema: HMS. user: root / 12345678
IDE (optional)	IntelliJ IDEA	All modules configured in .idea/modules.xml with Maven multi-module

10.2 Service Startup Order

Services must start in dependency order. Eureka must be available before any service attempts registration.

#	Service	Port	Start Command	Wait For
1	Eureka Server	8761	mvn spring-boot:run	Nothing — start first
2	Auth Service	8081	mvn spring-boot:run	Eureka on :8761
3	Hospital Service	8083	mvn spring-boot:run	Eureka on :8761
4	Patient Service	8082	mvn spring-boot:run	Hospital Service registered in Eureka
5	API Gateway	8080	mvn spring-boot:run	All services registered
6	Angular Frontend	4200	ng serve	API Gateway accepting requests

10.3 Key Configuration Reference

Service	Property	Value / Description
All services	eureka.client.service-url.defaultZone	http://localhost:8761/eureka/
Auth, Patient, Hospital	spring.datasource.url	jdbc:mysql://localhost:3306/HMS
Auth, Patient, Hospital	spring.datasource.username / password	root / 12345678
Auth, Patient, Hospital	spring.jpa.hibernate.ddl-auto	update (auto-creates tables on first run)
Auth Service	jwt.secret	256-bit hex string (MUST match API Gateway config)
Auth Service	jwt.access-token-expiration	86400000 ms = 24 hours
Auth Service	jwt.refresh-token-expiration	86400000 ms = 24 hours
API Gateway	jwt.secret	Must be identical to Auth Service jwt.secret
API Gateway	spring.cloud.gateway.routes	Array of route definitions (id, uri, predicates, filters)
Auth Service	server.port	8081
Patient Service	server.port	8082
Hospital Service	server.port	8083
Angular	API base URL (AuthService)	http://localhost:8080 (hardcoded, update for production)

10.4 Production Considerations



Recommended Changes Before Production Deployment

1. Externalise secrets: jwt.secret and DB passwords must come from environment variables or a secrets vault (e.g. HashiCorp Vault, AWS Secrets Manager) — never hardcoded in application.yaml.
2. Separate databases: Each microservice should own its own MySQL schema/database for true service independence.
3. HTTPS: All traffic should be TLS-terminated at the gateway or a load balancer in front of it.
4. Token expiry: Reduce access token expiry to 15 minutes and implement silent token refresh in Angular via HttpInterceptor.
5. Angular environment files: The API_URL should use Angular environment.ts — not hardcoded in AuthService.
6. Container deployment: Dockerise each service; use Docker Compose or Kubernetes for orchestration.

11. Glossary

Term	Definition
@PreAuthorize	Spring Security annotation on a controller method. Evaluated before method execution. E.g. @PreAuthorize('hasRole(DOCTOR)') blocks any non-doctor token with HTTP 403.
Aadhaar Number	India's 12-digit national identification number. Used in HMS as the primary patient lookup key for episode and appointment linking.
API Gateway	The single public entry point for all HTTP traffic. Validates JWTs, applies CORS, enriches requests with user-context headers, and routes to the correct microservice via Eureka.
BCrypt	Adaptive password hashing algorithm. Passwords are hashed with BCrypt before storage — the raw password is never persisted.
Consultation	The clinical record a doctor creates during an Encounter. Contains symptoms, diagnosis, notes. Linked to prescriptions and lab orders.
Encounter	A single clinical visit within an Episode, always paired with one Appointment. The Encounter is the container for the doctor's Consultation.
Episode	A clinical case grouping all Encounters for the same health concern. Has status ACTIVE (ongoing) or CLOSED (resolved). A patient can have multiple open episodes simultaneously.
Eureka	Netflix Service Registry. All Spring Boot services register their host:port here on startup. The API Gateway resolves service names (e.g. lb://PATIENT-SERVICE) to real addresses via Eureka.
Feign Client	Spring Cloud OpenFeign — a declarative REST client. Patient Service uses UserFeignClient to call Hospital Service for doctor name resolution without owning the users table.
hospitalId	Numeric tenant identifier embedded in the JWT. Forwarded as X-Hospital-Id header by the API Gateway. Every service-layer query filters by this value, making cross-hospital data access impossible.
JWT	JSON Web Token — a compact, URL-safe token signed with HMAC-SHA256. Carries all identity and role claims. Validated by the API Gateway without a database lookup.
LabOrder	A set of diagnostic tests ordered by a doctor. Progresses through ORDERED → BOOKED → SAMPLE_COLLECTED → IN_PROGRESS → COMPLETED. Can be CANCELLED before COMPLETED.
LabOrderTest	One individual test within a LabOrder (e.g. CBC, LFT). Each has its own status and stores the result value, unit, reference range, and remarks entered by the Lab Technician.

Term	Definition
Microservice	An independently deployable application with a single domain responsibility. HMS has four: Auth Service, Patient Service, Hospital Service, and API Gateway.
Multi-tenancy	The ability for a single instance of the application to serve multiple independent organisations (hospitals) with complete data isolation between them.
OncePerRequestFilter	A Spring servlet filter base class guaranteeing execution once per request. UserContextFilter extends this to rebuild the Spring SecurityContext from JWT-forwarded headers.
Sample	The physical specimen collected by a Lab Technician. Auto-assigned a barcode: LAB-{orderId}-{timestamp}. Sample type options include Blood, Urine, Stool, Swab, CSF, and others.
Spring Cloud Gateway	Reactive API gateway built on Project Reactor / WebFlux. Non-blocking — handles thousands of concurrent connections with a small thread pool.
Transactional	Spring @Transactional annotation. If any step in a multi-step database operation fails, all changes are rolled back atomically. Used on consultation submission and appointment creation.
UserContextFilter	OncePerRequestFilter in each microservice. Reads X-User-Id, X-Hospital-Id, X-Username, X-User-Roles headers and builds a Spring UsernamePasswordAuthenticationToken in the SecurityContext.
WebFlux	Spring's reactive web framework. The API Gateway uses WebFlux to process requests asynchronously — one thread handles many in-flight requests simultaneously.